CSD201 PE INSTRUCTIONS

Read the instructions below carefully before you start coding.

Students are ONLY allowed to use:

- Materials on their computer (including JDK, NetBeans...).
- For distance learning: Google Meet, Hangout (for Exam Monitoring Purpose).

Follow the steps below to complete PE:

1. Create a folder to store the given projects, e.g. CSD_given (1). Download the given materials into (1).
2. Complete the requirements of the test.
3. **Before submission:**
    - Clean and Build Project (Shift+F11)
    - Then rename the folder **dist** to **run**. (If the folder run already exists, delete it before renaming).
4. **Submission:**
    - To submit the project Q1, at first you must select the Question No: 1
    - Browse and select the project folder (e.g. Q1, or Q1A, or Q2, ...)
    - Then click the **Submit** button.
5. **Do not use accented Vietnamese** when writing comments in the programs.
6. **Do not add** new **import** statement(s) to the given files.
7. The software tools to be used are: **NetBeans IDE 8.x** and **Java JDK 1.8**. **If at least one of the above requirements is not followed, the exam will get ZERO.**

Notes:

- The input and expected output below are only for **testing your codes**.
- The input and expected output in the **real testcases** (for marking) are **different** from the examples below.
- **Do not hardcode** the given expected results.

Troubleshooting:

If the given project (e.g. Q1) runs with an error, run "Clean and Build Project" (Shift+F11). If the error persists, try renaming or copying the project to another name, e.g. from Q1 to Q1X or Q1Y.

Questions: (10 marks)

The given files already contain the code statements to implement a program that manages a Hotel. The main classes are structured as follows:

- Class **Room**: stores information about a Room object, including:
    - **code** (001, 002, 003, ...)
    - **status**: is 0 (if the Room is empty), or 1 (if the Room is occupied by customer(s))
    - **size** (must be > 0): the maximum number of people renting/using this Room
    - **price** (must be > 0)
- Class **Node**: holds **info** (a Room object) and **next** (a pointer for linking).
- Class **dataList**: is **a singly linked list** that manages regular Nodes with complete information. This class is responsible for managing all the Rooms in the Hotel.
- Class **requestQueue**: is **a queue** structure (implemented as a linked list), in which the **info** component of the Nodes **contains only size** (the number of people renting the Room) and **price** (the expected price, or the maximum money the customer(s) can pay). Note: **size>0 and price>0**.
- Class **Hotel**: is the main class of the program, containing a dataList that manages the Room objects and a requestQueue that represents a queue of orders.

Students must carefully read the provided code segments to fully understand the relationships between the classes and the functions inside each class. The specific task of the test is to implement the following requirements:

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*

a. f1(): 2.5 marks – Load data

To complete requirement f1(), students need to carry out two specific tasks: implement the function **addLast()** in the dataList class and the function **enQueue()** (similar to addLast) in the requestQueue class.

The expected output used to test your code is as follows:

```
Data List: (001,0,10,200) (003,0,3,70) (004,0,4,100) (005,0,3,70)
Request  : (1,100) (12,500) (4,50) (4,400)
```

b. f2(): 2.5 marks – Serve the first request in the requestQueue

To complete requirement f2(), students need to perform three specific tasks: (1) implement the **deQueue()** function of the requestQueue structure, (2) use the result returned by deQueue() to perform the **rent()** function, then (3) use these two functions to serve the first request in the body of f2().

The rent() action consists of two steps:

- First, **search** for the best Room to rent. If one is found, proceed to step 2. The best Room is:
  - An empty Room
  - Size >= the size in the request
  - Price <= the price in the request
  - If more than one Room satisfies these three conditions, choose the Room with the lowest price.
  - If more than one Room satisfies these four conditions, you should **choose the last** Room. For example: if Rooms 003, 005, and 007 satisfy the four conditions, the answer is Room 007.
- If the best Room is found, perform the renting action by **updating** the status in the dataList from 0 to 1.

The expected output used to test your code is as follows:

```
Data List: (001,0,10,200) (003,0,3,70) (004,0,4,100) (005,0,3,70)
Request  : (1,100) (12,500) (4,50) (4,400)
Data List: (001,0,10,200) (003,0,3,70) (004,0,4,100) (005,1,3,70)
Request  : (12,500) (4,50) (4,400)
```

Explanation:

- The request (1,100) is removed from the requestQueue, and then this request is processed.
- This request is for renting a room for one person (size=1), with the available money being 100 (price=100).
- The system finds two suitable rooms (empty (status=0), price <=100, lowest price 70), which are 003 and 005.
- The system then chooses room 005 because it appears after room 003 in the dataList. Finally, the status of this room is updated to status=1 to indicate that the room is now occupied.

c. f3(): 2.5 marks – Serve all requests in the requestQueue

Perform the pair of operations **deQueue()** and **rent()** for **all elements** in the **requestQueue**.

The expected output used to test your code is as follows:

```
Data List: (001,0,10,200) (003,0,3,70) (004,0,4,100) (005,0,3,70)
Request  : (1,100) (12,500) (4,50) (4,400)
Data List: (001,0,10,200) (003,0,3,70) (004,1,4,100) (005,1,3,70)
Request  : Empty
```

Explanation:

- As in f2(), the request (1,100) is served in room 005.
- The request (12,500) does not find a suitable Room because the number of people (size=12) exceeds the size of every available room.
- The request (4,50) also fails to find a room because it offers too little money (price=50), specifically less than the price of the cheapest room (price=70).
- The request (4,400) has two suitable rooms: 001 (price=200) and 004 (price=100), so room 004 is chosen because it has the lower price.
- As a result, rooms 004 and 005 are rented, and the requestQueue is empty.

*— Tài liệu được thành viên đăng tải tại **FUOVERFLOW** và được biên soạn lại để hỗ trợ ôn tập —*

d. f4(): 2.5 marks – Count the available Rooms after serve all requests in the requestQueue

You should: (1) perform the pair of operations deQueue() and rent() for all elements in the requestQueue, then (2) count the available Rooms with status = 0.

The expected output used to test your code is as follows:

```
Data List: (001,0,10,200) (003,0,3,70) (004,0,4,100) (005,0,3,70)
Request  : (1,100) (12,500) (4,50) (4,400)
Data List: (001,0,10,200) (003,0,3,70) (004,1,4,100) (005,1,3,70)
Request  : Empty
Available Room(s): 2
```

Explanation:
As in f3(), rooms 004 and 005 are rented. At that point, the Hotel has two available rooms: 001 and 003.

--- End ---